

Optimisation Convexe

Descentes de gradients

Écrit réflexif



Erwann Lesech

Mai 2025
Promotion 2026

Contents

1	Introduction	1
1.1	Préparation de l'analyse	1
1.1.1	Fonctions mathématiques	1
1.1.2	Approximation numérique de la dérivée	3
2	La descente de gradient	4
2.1	Avec un pas constant	4
2.1.1	Relation entre le pas et le nombre d'itérations	4
2.1.2	Lien entre la vitesse de convergence et le point initial	4
2.2	La descente de gradient avec backtracking	4
2.3	Choix de la norme : descente selon ℓ_1 ou ℓ_2	5
2.4	Sensibilité de la descente de gradient au conditionnement de la hessienne	6
2.5	Les méthodes d'accélération	6
2.5.1	Optimisation par Momentum	6
2.5.2	Optimisation de Nesterov	7
2.5.3	Optimisation Adam	7
2.6	Comparatif des méthodes d'accélération	8
3	La méthode de Newton	9
3.1	Introduction à la méthode de Newton	9
3.2	Comparaison avec les autres méthodes	9
4	Cas réel : Régression linéaire et polynomiale	10
4.1	Régression linéaire	10
4.2	Régression polynomiale	10
4.3	Vers le besoin de régularisation	11
5	Conclusion	12
6	Bibliographie	13
7	Annexes	14

Introduction

Cet écrit réflexif s'inscrit dans l'objectif de revenir sur le travail pratique destiné à l'analyse approfondie des implémentations des différentes méthodes de descentes de gradients. Nous souhaitons comprendre comment ces techniques s'articulent avec les notions théoriques abordées en cours, et en explorer les performances, les limitations ainsi que les conditions d'applicabilité. Ce TP a été structuré autour de plusieurs axes : l'implémentation naïve des descentes, le calcul du gradient, les méthodes d'optimisation comme le Momentum ou Adam, ainsi qu'une brève incursion dans la méthode de Newton.

Le TP nous a permis mettre en place les implémentations des différentes méthodes de descentes de gradients ainsi que des comparateurs entre ces différentes méthodes mais aussi entre les méthodes d'accélération (dits optimizers) et hyperparamètres pour comprendre, au mieux, le comportement de ces méthodes pour en extraire leurs avantages et inconvénients en fonction des fonctions abordées et des points initiaux utilisés.

1.1 Préparation de l'analyse

Avant de procéder aux expérimentations visant à comparer et comprendre le comportement des différentes méthodes de descente de gradient, il a été nécessaire de préparer rigoureusement l'environnement d'analyse. Cette préparation a consisté à définir les fonctions mathématiques sur lesquelles les algorithmes allaient être testés ; d'autre part, il a fallu mettre en place les mécanismes de calcul de dérivés qui sont fondamentaux pour le fonctionnement des descentes de gradients.

1.1.1 Fonctions mathématiques

Dans le cadre de ce travail pratique, l'analyse comparative des méthodes de descente de gradient a nécessité la conception d'un ensemble diversifié de fonctions tests. L'objectif était de confronter ces algorithmes à différents types de fonctions, afin d'observer leur comportement en fonction de la convexité, de la dimensionnalité et de la complexité des courbes d'optimisations. Les fonctions choisies se répartissent en deux catégories : les fonctions **convexes** et les fonctions **non convexes**. Nous avons exploré des cas unidimensionnels, bidimensionnels, puis progressivement étendu l'analyse à des fonctions de dimension supérieure (jusqu'à 20 dimensions), afin d'étudier l'impact de la dimension sur la performance des méthodes.

Fonctions convexes

Les fonctions convexes ont constitué une première base d'expérimentation, car elles garantissent l'existence d'un minimum local unique qui convient en tant que minimum global.

Deux types de fonctions convexes ont été utilisées :

- **Fonction quadratique :**

$$f(x) = \gamma x^2 + x + 1$$

Cette fonction, typique des problèmes convexes, a été utilisée dans des dimensions croissantes. Elle est centrée autour de l'origine, ce qui permet de connaître le minimum théorique à $x = 0$ pour toutes les dimensions. Cela facilite l'évaluation de la précision des méthodes entre le point optimal trouvé et le point optimal réel.

- **Fonction exponentielle symétrique :**

$$f(x) = \gamma (e^{\gamma x} - e^{-\gamma x})$$

Cette fonction présente une courbure variable en fonction de γ . Un faible γ donne lieu à une courbe plus étalée, nécessitant théoriquement un plus grand nombre d'itérations ou un pas d'apprentissage plus important pour converger efficacement. Son minimum global reste également situé en $x = 0$, ce qui facilite les comparaisons.

Ces fonctions nous ont permis d'analyser la stabilité des différentes méthodes de descente de gradient, leur vitesse de convergence, ainsi que leur sensibilité aux choix des hyperparamètres.

Fonctions non convexes

Afin d'évaluer la robustesse des méthodes dans des scénarios plus complexes, nous avons également travaillé avec des fonctions non convexes. Ces fonctions possèdent plusieurs minima locaux, rendant l'optimisation plus difficile et imprévisible. Cela permet d'identifier les algorithmes capables d'éviter ou de dépasser les minima locaux pour se rapprocher d'un minimum global.

Deux fonctions principales ont été utilisées :

- **Fonction cubique généralisée :**

$$f(x) = x^3 + \gamma x^2 + x + 1$$

Cette fonction introduit une non-linéarité et une asymétrie marquées, créant des minima locaux susceptibles de piéger les algorithmes naïfs ou mal paramétrés.

- **Fonction oscillatoire à puits multiples ("multi-sinks") :**

$$f(x) = 20 \cos(x^2) + \gamma x^2$$

Cette fonction combine un terme quadratique (qui stabilise globalement la fonction) avec un terme oscillatoire créant de nombreux puits locaux. Elle est particulièrement utile pour mettre en lumière les différences entre optimiseurs classiques (comme la descente de gradient simple) et ceux bénéficiant de mécanismes d'accélération ou de régularisation.

L'utilisation conjointe de fonctions convexes et non convexes nous a permis de réaliser une évaluation complète des méthodes testées, en mettant en évidence leur adaptabilité, leurs limites et les conditions dans lesquelles elles offrent les meilleures performances.

1.1.2 Approximation numérique de la dérivée

Pour estimer numériquement la dérivée d'une fonction $f : \mathbb{R} \rightarrow \mathbb{R}$, on utilise la définition suivante du nombre dérivé en un point $x \in \mathbb{R}$:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

En pratique, une petite valeur de h donne un quotient suffisamment proche du nombre dérivé que l'on cherche. La première méthode, dite **approche avant** [7] (forward difference), consiste à approximer la dérivée par :

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Cependant, cette méthode peut produire des erreurs importantes pour certaines fonctions. Par exemple, en évaluant cette approximation pour la fonction exponentielle en $x = 100$, l'erreur absolue obtenue est de l'ordre de 10^{37} .

Une méthode plus précise consiste à utiliser une approximation centrée (**centered difference**) [7], qui se formule ainsi :

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Cette méthode présente l'avantage d'une erreur d'ordre $o(h^2)$, contrairement à la méthode précédente dont l'erreur est d'ordre $o(h)$. Ainsi, pour la même fonction exponentielle en $x = 100$, l'erreur absolue obtenue avec cette méthode est réduite à l'ordre de 10^{34} , ce qui constitue une amélioration significative par rapport à la première approche.

Bien que cette erreur reste encore relativement grande, l'amélioration par rapport à la première méthode montre l'impact de l'approximation centrée sur la précision des résultats numériques.

La descente de gradient

2.1 Avec un pas constant

2.1.1 Relation entre le pas et le nombre d'itérations

Pour analyser la relation entre le pas et le nombre d'itérations, nous avons mené une étude en dimension 1, ce qui permet de visualiser plus facilement la descente de gradient sur un graphique. Nous avons choisi des valeurs de γ variant entre 1 et 10, afin de présenter des fonctions d'optimisation avec des pentes plus ou moins raides, et même des zones proches d'un plateau. L'observation principale est que plus le pas est faible, plus le nombre d'itérations nécessaires pour atteindre la convergence devient élevé. De plus, à mesure que γ augmente, l'importance du choix du pas devient encore plus significative. En effet, un pas trop grand risque d'empêcher l'algorithme de converger correctement, entraînant une explosion du nombre d'itérations.

Ainsi, le choix du pas influe directement sur le nombre d'itérations. Un pas optimisé permet une convergence rapide, tandis qu'un pas trop élevé ou trop faible peut ralentir le processus de convergence ou, dans le pire des cas, empêcher l'algorithme d'atteindre un résultat convergent.

2.1.2 Lien entre la vitesse de convergence et le point initial

Pour comprendre à quel point le point initial influe sur la vitesse de convergence, nous prenons une même fonction de dimension 1, que nous confrontons à la même descente de gradient simple, mais avec 1000 points initiaux différents, répartis entre 1 et 10 000, afin d'observer des vitesses de convergence divergentes.

Malheureusement, cette première analyse a démontré qu'en dimension 1, le terrain d'optimisation est probablement trop "simple", et que le choix du point initial n'a que peu d'impact sur la vitesse de convergence. Néanmoins, cet aspect demeure important à garder à l'esprit, car nous verrons par la suite qu'il devient significatif en fonction des optimisateurs utilisés, tels qu'Adam.

2.2 La descente de gradient avec backtracking

Le choix du pas de descente dans le cadre d'une descente de gradient à pas constant est un élément crucial pour garantir la convergence de l'algorithme. Cependant, ce choix peut s'avérer délicat et ne pas être optimal pour toutes les fonctions, car un pas trop petit ralentit la convergence, tandis qu'un pas trop grand peut provoquer la divergence.

Pour pallier cette difficulté, nous explorons ici l'utilisation du *backtracking*, une méthode adaptative qui ajuste dynamiquement le pas à chaque itération. Ce critère

permet d'assurer une diminution suffisante de la fonction objectif, et rend ainsi l'algorithme bien moins sensible aux choix initiaux du pas γ , contrairement à la méthode à pas constant. L'inconvénient principal du backtracking réside dans son coût computationnel par itération, dû aux multiples évaluations de f nécessaires pour ajuster le pas.

En comparant les performances des deux approches sur plusieurs fonctions, on constate que le backtracking permet généralement de réduire le nombre d'itérations pour atteindre la convergence, ce qui accélère le processus global d'optimisation malgré un temps de calcul unitaire plus élevé. Sa robustesse face à des choix de pas initiaux variés constitue un avantage non négligeable dans des contextes où l'estimation d'un pas optimal est difficile.

Nous avons néanmoins choisi de ne pas approfondir cette analyse, préférant nous concentrer sur d'autres paramètres pouvant également influencer la descente de gradient. Il aurait été pertinent de comparer également les temps d'exécution des deux méthodes. Les graphiques comparatifs des performances sur quatre fonctions mathématiques en dimension 1 sont disponibles en annexe ([cf. annexe 1](#)).

2.3 Choix de la norme : descente selon ℓ_1 ou ℓ_2

En dimension 1, comparer la descente de gradient classique à celle en norme ℓ_1 n'a pas de sens, car il n'existe qu'une seule direction possible, rendant les deux approches équivalentes. En revanche, en dimension 2 ou plus, le choix de la norme influe directement sur la direction de descente. La descente classique (norme ℓ_2) suit le vecteur du gradient complet, tandis que la descente selon la norme ℓ_1 privilégie la direction de la coordonnée avec la plus forte dérivée partielle en valeur absolue, ce qui produit un comportement par "marches" successives sur les axes.

Cette stratégie conduit à un parcours en escalier vers le minimum, contrairement à la trajectoire plus lisse de la descente classique. Cela se reflète dans les résultats expérimentaux : bien que les deux méthodes convergent vers le même point optimal, la descente ℓ_1 nécessite généralement plus d'itérations. Par exemple, avec la fonction quadratique de dimension 2, pour un pas $\gamma = 5.0$, la descente classique converge en 69 itérations contre 70 pour la version ℓ_1 , mais pour $\gamma = 2.0$, l'écart monte à 69 contre 102. De plus, on observe que la descente ℓ_1 est plus sensible à l'augmentation de γ , ce qui peut provoquer un nombre d'itérations bien plus élevé, voire une non-convergence.

Ainsi, même si la descente ℓ_1 peut être utile dans certains contextes (ex. : optimisation sous contraintes ce qui n'est pas l'objet de cette étude), la descente classique reste plus efficace en termes de rapidité de convergence dans des cas standards.

2.4 Sensibilité de la descente de gradient au conditionnement de la hessienne

Pour comprendre la sensibilité de la descente de gradient au conditionnement de la hessienne, on rappelle que la hessienne d'une fonction f est la matrice carrée des dérivées secondes, notée $\nabla^2 f(x)$, qui décrit la courbure locale de la fonction. Le nombre de conditionnement de cette matrice, défini comme le rapport entre la plus grande et la plus petite valeur propre positive, mesure l'anisotropie locale du paysage de l'erreur. Un nombre de conditionnement élevé indique une grande différence de courbure selon les directions, ce qui peut ralentir la convergence de la descente de gradient.

Pour notre étude, on distingue ici deux cas : la descente de gradient à pas constant et celle avec backtracking. On limite l'étude aux fonctions dont le nombre de conditionnement de la hessienne dépend de γ , en particulier celles de dimensions supérieures à 2.

On observe alors que, pour la descente à pas constant, lorsque le pas dépasse $\frac{1}{\gamma}$, l'algorithme ne converge plus et peut même diverger avec des erreurs numériques (overflow, NaN), ce qui est cohérent avec les résultats théoriques du cours. En revanche, avec le backtracking, ce problème ne se pose pas car le pas est ajusté dynamiquement à chaque itération. Cela permet d'adapter la descente aux variations locales de la géométrie de la fonction, et donc de mieux gérer les situations où le conditionnement est élevé. Ainsi, l'impact du conditionnement sur la vitesse de convergence est significativement réduit avec cette seconde méthode. Les deux résultats graphiques sont disponibles en annexe ([cf. annexe 2](#)).

2.5 Les méthodes d'accélération

Nous avons expérimenté 3 méthodes d'accélération :

- Momentum Optimisation [5]
- Nesterov Optimisation [3]
- Adamn Optimisation [4]

2.5.1 Optimisation par Momentum

La première méthode d'accélération que nous avons étudiée est l'optimisation par momentum. Elle consiste à intégrer un effet d'inertie dans la mise à jour des points. Plutôt que de simplement avancer dans la direction opposée du gradient pondéré par le pas d'apprentissage, on ajoute un terme appelé *vitesse*, qui tient compte des gradients précédents. Cela permet de lisser les mises à jour, d'accélérer la convergence dans les vallées étroites et de réduire le risque de rester bloqué dans des minima locaux. [2]

Les mises à jour se font selon les équations suivantes :

$$v_t = \beta v_{t-1} - \eta \nabla f(x_{t-1})$$

$$x_t = x_{t-1} + v_t$$

Lors des expérimentations sur la fonction quadratique de dimension 2, on remarque que cette méthode permet d'atteindre plus rapidement les zones proches du minimum par rapport à la descente classique, notamment au début de l'optimisation.

2.5.2 Optimisation de Nesterov

La méthode de Nesterov est une amélioration du momentum classique. L'idée principale est d'anticiper la direction dans laquelle on va se déplacer, en effectuant une estimation du point futur avant de calculer le gradient. Autrement dit, le gradient est évalué non pas au point actuel, mais à un point "look-ahead", ce qui permet une meilleure correction de trajectoire.

Les équations de mise à jour sont :

$$v_t = \beta v_{t-1} - \eta \nabla f(x_{t-1} + \beta v_{t-1})$$

$$x_t = x_{t-1} + v_t$$

Ce mécanisme réduit le risque de dépasser le minimum et permet en général une convergence plus stable et plus rapide. Comme pour le momentum classique, les premières étapes sur la fonction quadratique montrent une amélioration sensible par rapport à la descente de gradient standard.

2.5.3 Optimisation Adam

Enfin, nous avons testé l'optimiseur Adam, une méthode plus sophistiquée qui combine les avantages du momentum et de la stratégie adaptative de type RMSprop. Elle suit une double estimation : une moyenne mobile des gradients (comme le momentum) et une moyenne mobile de leur carré (comme RMSprop). Ces estimations sont ensuite corrigées pour compenser leur biais initial.

Les mises à jour suivent les formules suivantes :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(x_t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla f(x_t))^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$x_{t+1} = x_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Nos expériences sur la fonction quadratique, Adam n'a pas donné de meilleurs résultats que les méthodes précédentes. Pourtant Adam est aujourd'hui largement utilisé dans le domaine du machine learning, notamment pour l'optimisation de fonctions non convexes, grâce à sa robustesse et sa rapidité de convergence. Cela montre que ses performances peuvent dépendre fortement du type de fonction optimisée. C'est pourquoi nous avons fait un comparatif complet en fin d'expérimentation pour confronter ces différentes méthodes d'accélération à nos 4 fonctions témoins dont 2 ne sont pas convexes.

2.6 Comparatif des méthodes d'accélération

Pour comparer les performances des différentes méthodes d'optimisation, nous avons appliqué chaque algorithme sur l'ensemble des fonctions tests définies en début de projet. Toutes les expériences ont été réalisées avec des pas constants, en faisant varier plusieurs paramètres : le nombre de conditionnement γ (entre 1 et 10), le pas d'apprentissage (entre 0,01 et 0,5), ainsi que plusieurs points de départ autour du minimum théorique.

Les performances ont été évaluées principalement en fonction de la distance finale au minimum attendu, du nombre d'itérations nécessaires à la convergence, et du temps d'exécution. Chaque configuration a été testée en excluant les cas où le pas choisi dépassait la borne théorique $\frac{1}{\gamma}$, afin d'assurer la convergence dans le cadre de la descente de gradient classique.

Les résultats mettent en évidence plusieurs tendances :

- **Sur les fonctions quadratiques convexes**, les performances les plus constantes sont obtenues avec la descente de gradient classique, c'est-à-dire sans accélération. Dans ces cas, les directions de descente sont bien définies et les conditions favorisent une convergence rapide, même sans techniques sophistiquées.
- **Les méthodes basées sur le momentum** (classique ou Nesterov) n'ont pas surpassé la méthode standard sur ces fonctions, probablement en raison de l'efficacité déjà élevée de la descente classique dans ce cadre fortement convexe. Il est également possible que les paramètres d'inertie (β) n'aient pas été ajustés de manière optimale, faute de temps pour un réglage fin.
- **Sur les fonctions non convexes**, telles que cubique et multi-sinks, les approches adaptatives montrent un avantage. En particulier, **l'optimiseur Adam** obtient de meilleurs résultats, atteignant plus fréquemment des minima proches de l'optimum global. Cela confirme la robustesse d'Adam dans des contextes complexes, comme le soulignait déjà la littérature.
- Enfin, notons que la **variabilité des résultats selon les points de départ** est notable pour toutes les méthodes. Cela rappelle l'importance du choix des conditions initiales, en particulier pour les fonctions non convexes où les algorithmes peuvent converger vers des minima locaux.

Ainsi, nos expérimentations confirment les propriétés théoriques des méthodes. Les optimisateurs simples suffisent pour les fonctions bien conditionnées et convexes, tandis que des méthodes plus avancées comme Adam sont mieux adaptées à des paysages d'erreur plus complexes. Bien que nous n'ayons pas exploré tous les réglages d'hyperparamètres possibles, les résultats obtenus offrent un aperçu pertinent de leurs comportements respectifs.

La méthode de Newton

3.1 Introduction à la méthode de Newton

La méthode de Newton [6] est une approche d'optimisation qui va au-delà de la simple descente de gradient en utilisant la matrice Hessienne pour déterminer la direction de descente. La méthode de Newton permet une convergence plus rapide que la descente de gradient classique, car elle prend en compte non seulement la pente (le gradient), mais aussi la courbure de la fonction à optimiser. La matrice Hessienne, qui est la matrice des dérivées secondes de la fonction, fournit cette information cruciale.

La mise à jour de la position à chaque itération dans la méthode de Newton se fait de la manière suivante :

$$x_{k+1} = x_k - H^{-1}(x_k) \nabla f(x_k)$$

où x_k est la position actuelle, $H(x_k)$ est la matrice Hessienne de la fonction f à la position x_k , et $\nabla f(x_k)$ est le gradient de la fonction f à cette position. L'inverse de la Hessienne $H^{-1}(x_k)$ est utilisé pour ajuster la direction de descente, prenant en compte la courbure de la fonction.

En comparaison avec la descente de gradient classique [1], qui met à jour la position selon :

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

où α_k est un facteur d'apprentissage, la méthode de Newton utilise une information de second ordre (la Hessienne) pour mieux adapter la taille du pas et la direction de descente. Cela permet une convergence beaucoup plus rapide, particulièrement lorsque la fonction est bien conditionnée et de faible dimension.

Cependant, la méthode de Newton est plus coûteuse en calcul, car elle nécessite de calculer la Hessienne et son inverse à chaque itération, ce qui peut être prohibitif pour des problèmes de grande dimension.

3.2 Comparaison avec les autres méthodes

La comparaison entre la méthode de Newton et les autres méthodes de descente, comme la descente de gradient classique, peut être effectuée en mesurant la distance entre le point optimal calculé et le point optimal réel. Sur les fonctions de test utilisées, la méthode de Newton s'est révélée supérieure à toutes les autres méthodes en termes de précision et de rapidité de convergence.

Néanmoins, la méthode de Newton présente un coût computationnel élevé, surtout pour les fonctions de grande dimension, car le calcul et l'inversion de la Hessienne peuvent être lourds. Pour des problèmes à grande échelle, des variantes comme la méthode de Newton modifiée ou des approches quasi-Newton (qui approximerait la Hessienne) sont souvent préférées.

Cas réel : Régression linéaire et polynomiale

Pour conclure ce TP, nous avons mis en œuvre la descente de gradient pour résoudre un problème de régression supervisée. Deux types de modèles ont été étudiés : la régression linéaire simple, puis la régression polynomiale.

4.1 Régression linéaire

Le modèle de régression linéaire consiste à ajuster une droite aux données d'entraînement. L'objectif est de trouver les poids optimaux θ qui minimisent l'erreur quadratique moyenne (MSE, pour *Mean Squared Error*), définie par la formule suivante :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{avec} \quad \hat{y}_i = \theta^\top x_i$$

où :

- n est le nombre d'échantillons dans le jeu de données,
- y_i est la vraie valeur de sortie,
- $\hat{y}_i$ est la valeur prédite par le modèle,
- x_i est l'entrée

Grâce à l'utilisation de la descente de gradient, combinée à une stratégie de *backtracking* pour ajuster dynamiquement le taux d'apprentissage, nous avons réussi à obtenir une estimation des poids très proche de la solution analytique issue de la pseudo-inverse, ce qui confirme la convergence correcte de notre implémentation.

4.2 Régression polynomiale

Dans un second temps, nous avons étendu notre modèle à une régression polynomiale. Cela consiste à enrichir l'espace des caractéristiques en ajoutant des puissances successives de chaque variable d'entrée, par exemple :

$$x \rightarrow [x, x^2, x^3, \dots, x^d]$$

Cette transformation permet au modèle d'approximer des relations non linéaires entre les variables indépendantes et la variable cible. Le processus d'optimisation reste identique (minimisation de la MSE via descente de gradient), mais appliqué dans un espace de dimension supérieure.

4.3 Vers le besoin de régularisation

L'ajout de termes polynomiaux augmente la capacité du modèle, mais également le risque de surapprentissage (*overfitting*). Ce phénomène apparaît lorsque le modèle s'ajuste trop fidèlement aux données d'entraînement, au détriment de sa capacité de généralisation.

C'est ici que la régularisation trouve tout son intérêt. En ajoutant un terme pénalisant la complexité du modèle, tel que la norme des poids, on cherche à contrôler la taille des coefficients et ainsi éviter les oscillations extrêmes du polynôme ajusté.

Un exemple courant est la régularisation de type Ridge (ou L2), qui modifie la fonction de coût comme suit :

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \|\theta\|^2$$

où $\lambda > 0$ est un hyperparamètre contrôlant la force de la pénalisation. Une autre variante, la régularisation Lasso (L1), pénalise la somme des valeurs absolues des coefficients, favorisant des solutions creuses.

Dans notre étude, nous n'avons pas observé de nécessité immédiate de régularisation, sans doute à cause de la taille modeste des données et du bon conditionnement du problème. Toutefois, dans des cas plus complexes ou bruités, ou lorsque le degré du polynôme est élevé, la régularisation devient essentielle pour obtenir un modèle robuste.

Conclusion

Au cours de ce travail, nous avons démontré l'importance du choix du pas d'apprentissage (γ) pour assurer la convergence rapide des algorithmes. L'utilisation de techniques comme le *backtracking* permet de surmonter cette difficulté en ajustant dynamiquement le pas.

Nous avons également analysé l'impact du choix de la norme utilisée pour la descente, montrant que la norme ℓ_2 offre généralement des performances plus stables, bien que la norme ℓ_1 puisse être utile dans des contextes spécifiques, comme l'optimisation sous contraintes ou la régularisation. De plus, la sensibilité de la descente de gradient au conditionnement de la hessienne a été démontrée, mettant en avant le *backtracking*, permettent de mieux gérer ces situations.

Les méthodes d'accélération, telles que le *momentum*, l'optimisation de *Nesterov* et *Adam*, ont également été examinées dans le cadre de notre étude. Bien que ces méthodes offrent des avantages significatifs dans des problèmes non convexes, où elles aident à éviter les minima locaux et accélèrent la convergence, leur utilisation dans des problèmes convexes simples, comme les fonctions quadratiques, n'a pas montré de bénéfices importants par rapport à la descente de gradient classique. En effet, en raison de la simplicité de la fonction quadratique, la descente de gradient classique suffit à garantir une convergence rapide et efficace. Cependant, dans des contextes plus complexes et des problèmes à plus haute dimension, ces méthodes d'accélération peuvent s'avérer cruciales pour optimiser la vitesse de convergence et surmonter les défis posés par des minima locaux ou des fonctions de coût plus complexes.

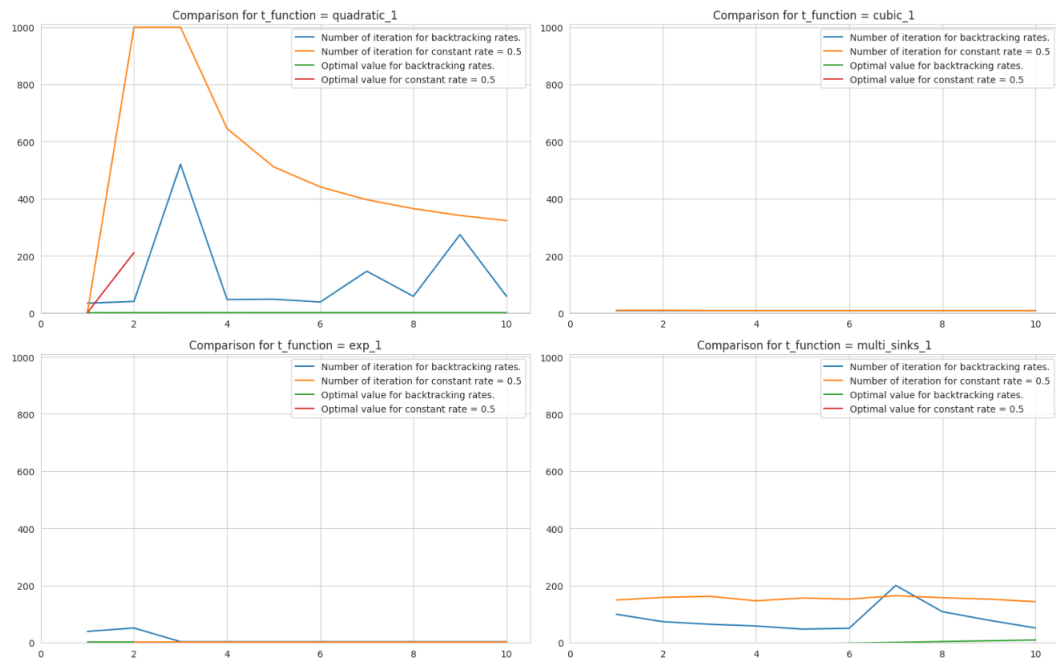
La méthode de *Newton* a été présentée comme une approche plus avancée, apportant une convergence rapide grâce à l'utilisation de la matrice Hessienne. Toutefois, son coût computationnel élevé en fait une méthode réservée aux problèmes de petite dimension, avec des variantes comme les méthodes quasi-Newton qui offrent un compromis entre efficacité et coût.

Dans l'ensemble, ce travail met en évidence la diversité des méthodes d'optimisation disponibles et leur influence sur la vitesse de convergence et la précision des solutions obtenues. L'expérimentation des méthodes avancées et leur comparaison directe fournit une meilleure compréhension de leur performance dans différents contextes, offrant ainsi des pistes pour des optimisations futures.

Bibliographie

- [1] Baeldung. *gradient descent vs newtons gradient descent*. <https://www.baeldung.com/cs/gradient-descent-vs-newtons-gradient-descent>.
- [2] Visually explained. *Momentum Gradient Descent*. https://www.youtube.com/watch?app=desktop&v=iudXf5n_3ro.
- [3] jlmelville. *Nesterov*. <https://jlmelville.github.io/mize/nesterov.html>.
- [4] Medium. *How to implement an adam optimizer from scratch*. <https://medium.com/the-ml-practitioner/how-to-implement-an-adam-optimizer-from-scratch-76e7b217f1cc>.
- [5] stmorse. *Momentum vs Acceleration*. <https://stmorse.github.io/journal/Momentum-vs-Acceleration.html>.
- [6] vdeborto. *cours_{optim}*. https://vdeborto.github.io/teaching/optimization/cours_optim/TP/tp1/tp1_2017.pdf.
- [7] Wikipedia. *Finite Difference*. https://en.wikipedia.org/wiki/Finite_difference.

Annexe 1 - Comparaison backtracking



Annexe 2 - Relation nombre de conditionnements et nombres d'itérations

